МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
**«Сибирский государственный университет науки и технологий
имени академика М.Ф. Решетнева»**

Институт информатики и телекоммуникаций

Кафедра информатики и вычислительной техники

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

Алгоритмы и структуры данных

Лабораторная работа №4. Бинарные деревья

Руководитель _____ В.В. Тынченко
подпись, дата инициалы, фамилия

Обучающийся БПИ24-02, 24151430 _____ В.Т. Гордов
номер группы, зачетной книжки подпись, дата инициалы, фамилия

Красноярск 2025 г.

ЦЕЛЬ РАБОТЫ

Изучение метода рекурсивной обработки данных; знакомство с принципами организации и обработки бинарных деревьев.

ПОСТАНОВКА ЗАДАЧИ

Выполнить задание в соответствии с вариантом. Для решения поставленной задачи разработать и использовать шаблон класса «Бинарное дерево поиска».

Для оценки «удовлетворительно».

Шаблон класса «Бинарное дерево поиска» должен включать в себя необходимый минимум методов, обеспечивающий полноценное функционирование объектов указанного класса, а именно:

- конструктор по умолчанию;
- конструктор копирования;
- деструктор;
- добавление узла;
- удаление узла;
- поиск узла с заданным значением ключа;

Используя механизм перегрузки стандартных операций, реализовать:

- вывод дерева на экран в удобном для восприятия человеком виде;
- ввод данных в дерево с клавиатуры;
- сравнение двух деревьев;
- присваивание данных одного дерева другому дереву;
- решение задачи согласно варианту задания.

Для оценки «хорошо».

Дополнить разработанный класс следующими функциональными возможностями:

- прямой обход дерева (корень, левое поддерево, правое поддерево);
- обратный обход дерева (левое поддерево, корень, правое поддерево);
- концевой обход дерева (левое поддерево, правое поддерево, корень).
- вывод дерева в файл в удобном для восприятия человеком виде;
- ввод данных в дерево из файла;

Для оценки «отлично».

На базе разработанного класса «Бинарное дерево поиска», реализовать производный класс «Сбалансированное бинарное дерево», добавив показатель балансировки для узлов дерева и метод, выполняющий при необходимости повторную балансировку дерева.

Вариант №5.

Из входной последовательности вещественных чисел построить бинарное дерево поиска. Вывести на экран значения в тех узлах, которые имеют уровень, заданный пользователем.

ХОД РАБОТЫ

В ходе лабораторной работы была разработана программа, предназначенная для изучения принципов организации и обработки бинарных деревьев.

Была реализована структура **Node**, предназначенная для хранения данных узла дерева (Рисунок 1).

```
// Шаблон узла дерева
You, 20 часов назад | 1 author (You)
template <typename T>
struct Node {
    T key; // Значение ключа
    Node* left; // Указатель на левое поддерево
    Node* right; // Указатель на правое поддерево
    int height; // Высота узла

    // Конструктор узла
    Node(T k) : key(k), left(nullptr), right(nullptr), height(1) {}
};
```

Рисунок 1 — Структура Node

Был реализован класс **Tree**, представляющий собой бинарное дерево поиска (Рисунок 2).

```
// Шаблон класса бинарного дерева поиска
You, 39 минут назад | 1 author (You)
template <typename T>
class Tree {
protected:
    Node<T>* root; // Корень дерева
```

Рисунок 2 —Класс Tree

Данный класс обеспечивает следующий функционал:

- создание и удаление дерева;
- добавление и удаление узлов;
- поиск узлов;
- прямой, обратный и концевой обходы дерева;
- ввод данных дерева с клавиатуры и из файла;
- вывод данных дерева в консоль и файл;
- поиск и отображение узлов с заданного пользователем уровня.

Был реализован набор методов, обеспечивающих полноценное функционирование объектов указанного класса при их использовании в программе (Рисунки 3-8).

```
31      // Рекурсивное копирование дерева
32      Node<T>* copy(Node<T>* node) {
33          if (!node)
34              return nullptr;
35          Node<T>* newNode = new Node<T>(node->key);
36          newNode->left = copy(node->left);
37          newNode->right = copy(node->right);
38          newNode->height = node->height;
39          return newNode;
40      }
41
42      // Очистка дерева
43      void clear(Node<T>* node) {
44          if (!node)
45              return;
46          clear(node->left);
47          clear(node->right);
48          delete node;
49      }
50
51      // Добавление узла
52      virtual Node<T>* add(Node<T>* node, T key) {
53          if (!node)
54              return new Node<T>(key);
55          if (key < node->key)
56              node->left = add(node->left, key);
57          else
58              node->right = add(node->right, key);
59          return node;
60      }
61
62      // Поиск узла по ключу
63      Node<T>* search(Node<T>* node, T key) const {
64          if (!node)
65              return nullptr;
66          if (node->key == key)
67              return node;
68          if (key < node->key)
69              return search(node->left, key);
70          else
71              return search(node->right, key);
72      }
73
74      // Поиск минимального узла
75      Node<T>* findMin(Node<T>* node) {
76          while (node && node->left)
77              node = node->left;
78          return node;
79      }
80
81      // Удаление узла по ключу
82      Node<T>* remove(Node<T>* node, T key) {
83          if (!node)
84              return nullptr;
85          if (key < node->key)
86              node->left = remove(node->left, key);
87          else if (key > node->key)
88              node->right = remove(node->right, key);
89          else { // Нужный узел найден.
90              if (!node->left) {
```

Рисунок 3 — Методы класса Tree – часть 1

```
89          else { // Нужный узел найден.
90              if (!node->left) {        You, вчера • Прототип 4 лабы
91                  Node<T>* temp = node->right;
92                  delete node;
93                  return temp;
94              }
95              else if (!node->right) {
96                  Node<T>* temp = node->left;
97                  delete node;
98                  return temp;
99              }
100             // Узел имеет двух потомков. Находим минимальный элемент в правом поддереве.
101             Node<T>* temp = findMin(node->right);
102             node->key = temp->key; // Копируем значение
103             node->right = remove(node->right, temp->key); // Удаляем дубликат
104         }
105         return node;
106     }
107
108     // Обходы дерева
109
110     // Прямой (префиксный) обход дерева
111     // Корень, левое поддерево, правое поддерево
112     void pre(Node<T>* node) const {
113         if (!node)
114             return;
115         cout << node->key << " ";
116         pre(node->left);
117         pre(node->right);
118     }
119
120     // Обратный (инфиксный) обход дерева
121     // Левое поддерево, корень, правое поддерево
122     void in(Node<T>* node) const {
123         if (!node)
124             return;
125         in(node->left);
126         cout << node->key << " ";
127         in(node->right);
128     }
129
130     // Концевой (постфиксный) обход дерева
131     // Левое поддерево, правое поддерево, корень
132     void post(Node<T>* node) const {
133         if (!node)
134             return;
135         post(node->left);
136         post(node->right);
137         cout << node->key << " ";
138     }
139
140     // Вывод дерева в поток
141     void print(Node<T>* node, int level, ostream& os, T parentKey = T{}) const {
142         if (!node)
143             return;
144
145         // Печатаем текущий узел с указанием уровня и родителя
146         if (parentKey == T{}) {
147             os << "Root: " << node->key << endl;
```

Рисунок 4 — Методы класса Tree – часть 2

```
146         if (parentKey == T{}) {
147             os << "Root: " << node->key << endl;        You, вчера • Прототип 4 лабы
148         }
149         else {
150             os << "node: " << node->key << " (Parent: " << parentKey << ")" << endl;
151         }
152 
153         // Рекурсивно печатаем левое поддерево
154         if (node->left) {
155             os << string((level + 1) * 2, ' ') << "Left ";
156             print(node->left, level + 1, os, node->key);
157         }
158 
159         // Рекурсивно печатаем правое поддерево
160         if (node->right) {
161             os << string((level + 1) * 2, ' ') << "Right ";
162             print(node->right, level + 1, os, node->key);
163         }
164     }
165 
166     // Сбор всех значений узлов в вектор
167     void collectValues(Node<T>* node, vector<T>& vals) {
168         if (!node)
169             return;
170         vals.push_back(node->key);
171         collectValues(node->left, vals);
172         collectValues(node->right, vals);
173     }
174 
175     // Сбор узлов, находящихся на заданном уровне, в вектор
176     void collectAtLevel(Node<T>* node, int targetLevel, int currentLevel, vector<T>& result) {
177         if (!node)
178             return;
179         if (currentLevel == targetLevel) {
180             result.push_back(node->key);
181         }
182         collectAtLevel(node->left, targetLevel, currentLevel + 1, result);
183         collectAtLevel(node->right, targetLevel, currentLevel + 1, result);
184     }
185 
186     // Сравнение структуры и данных двух деревьев
187     bool compare(Node<T>* a, Node<T>* b) const {
188         if (!a && !b)
189             return true;
190         if (!a || !b)
191             return false;
192         return (a->key == b->key) &&
193                 compare(a->left, b->left) &&
194                 compare(a->right, b->right);
195     }
196 
197     // Сохранение дерева в файл (префиксный порядок)
198     void saveHelper(Node<T>* node, ofstream& out) {
199         if (!node)
200             return;
201         out << node->key << " ";
202         saveHelper(node->left, out);
203         saveHelper(node->right, out);
```

Рисунок 5 — Методы класса Tree – часть 3

```
203         saveHelper(node->right, out);        You, вчера • Прототип 4 лаб
204     }
205
206 public:
207     // Конструктор по умолчанию
208     Tree() : root(nullptr) {}
209
210     // Конструктор копирования
211     Tree(const Tree& other) {
212         root = copy(other.root);
213     }
214
215     // Виртуальный деструктор
216     virtual ~Tree() {
217         clear(root);
218     }
219
220     // Оператор присваивания
221     Tree& operator=(const Tree& other) {
222         if (this != &other) {
223             clear(root);
224             root = copy(other.root);
225         }
226         return *this;
227     }
228
229     // Оператор сравнения
230     bool operator==(const Tree& other) const {
231         return compare(root, other.root);
232     }
233
234     // Оператор вывода
235     friend ostream& operator<<(ostream& os, const Tree<T>& tree) {
236         tree.print(tree.root, 0, os);
237         return os;
238     }
239
240     // Оператор ввода
241     friend istream& operator>>(istream& is, Tree<T>& tree) {
242         T val;
243         cout << "Enter value to add: ";
244         if (is >> val) {
245             tree.addValue(val);
246         }
247         return is;
248     }
249
250     // Добавить узел
251     void addValue(T key) {
252         root = add(root, key);
253     }
254     // Удалить узел
255     void removeValue(T key) {
256         root = remove(root, key);
257     }
258     // Найти узел
259     bool searchValue(T key) const {
260         return search(root, key) != nullptr;
261     }
```

Рисунок 6 — Методы класса Tree – часть 4

```
263     // Визуальный вывод дерева
264     void printTree() const {
265         print(root, 0, cout);
266     }
267
268     // Прямой (префиксный) обход дерева
269     // Корень, левое поддерево, правое поддерево
270     void preOrder() const { pre(root); cout << endl; }
271     // Обратный (инфиксный) обход дерева
272     // Левое поддерево, корень, правое поддерево
273     void inOrder() const { in(root); cout << endl; }
274     // Концевой (постфиксный) обход дерева
275     // Левое поддерево, правое поддерево, корень
276     void postOrder() const { post(root); cout << endl; }
277
278     // Загрузка в файл
279     void saveToFile(const string& filename) {
280         ofstream out(filename);
281         if (!out.is_open()) {
282             cout << "Error opening file for writing.\n";
283             return;
284         }
285         saveHelper(root, out);
286         out.close();
287         cout << "Tree saved to file " << filename << endl;
288     }
289
290     // Загрузка из файла
291     void loadFromFile(const string& filename) {
292         ifstream in(filename);
293         if (!in.is_open()) {
294             cout << "Error opening file for reading.\n";
295             return;
296         }
297         T val;
298         clear(root);
299         root = nullptr;
300         while (in >> val) {
301             addValue(val);
302         }
303         in.close();
304         cout << "Tree loaded from file " << filename << endl;
305     }
306
307     // ЗАДАНИЕ 5 ВАРИАНТА
308     // Выводит значений узлов с заданного уровня
309     void executeVariantTask() {
310         if (!root) {
311             cout << "Tree is empty.\n";
312             return;
313         }
314
315         // Запрашиваем у пользователя уровень
316         int targetLevel;
317         cout << "Enter the level to display nodes from: ";
318         cin >> targetLevel;
319
320         // Проверяем корректность ввода
321         if (targetLevel < 1) {
322             cout << "Level must be >= 1.\n";
```

Рисунок 7 — Методы класса Tree – часть 5

```
322            cout << "Level must be >= 1.\n";        You, вчера • Прототип
323            return;
324        }
325
326        // Собираем все значения узлов, находящихся на заданном уровне
327        vector<T> nodesAtLevel;
328        collectAtLevel(root, targetLevel, 1, nodesAtLevel);
329
330        // Выводим результат
331        cout << "\n--- Nodes at Level " << targetLevel << " ---\n";
332        if (nodesAtLevel.empty()) {
333            cout << "No nodes found at this level.\n";
334        }
335        else {
336            for (const auto& val : nodesAtLevel) {
337                cout << val << " ";
338            }
339            cout << endl;
340        }
341    }
342 };
```

Рисунок 8 — Методы класса Tree – часть 6

В ***public*** фрагменте класса **Tree** реализованы методы, отвечающие за взаимодействие пользователя с объектом бинарного дерева (Рисунки 6-8).

Был реализован дочерний класс **AVL** на основе класса **Tree**, представляющий собой сбалансированное бинарное дерево (Рисунки 9-10).

Дополнительные методы данного класса предлагают следующий функционал:

- расчёт высоты узла;
- расчёт коэффициента баланса узла;
- «повороты» узла влево и вправо;
- добавление узла дерева с последующей балансировкой.

```
// Сбалансированное бинарное дерево (AVL)
You, 3 часа назад | 1 author (You)
template <typename T>
class AVL : public Tree<T> {
    using Tree<T>::root; // Доступ к root из базового класса

    // Получение высоты узла
    int height(Node<T>* node) {
        return node ? node->height : 0; }

    // Вычисление фактора баланса узла
    // (Разность высот левого и правого поддеревьев)
    int balance(Node<T>* node) {
        return node ? height(node->left) - height(node->right) : 0; }

    // Правый поворот
    Node<T>* rotateRight(Node<T>* y) {
        // Меняем местами узлы
        Node<T>* x = y->left;
        Node<T>* T2 = x->right;
        x->right = y;
        y->left = T2;
        // Пересчитываем высоты
        y->height = max(height(y->left), height(y->right)) + 1;
        x->height = max(height(x->left), height(x->right)) + 1;
        return x;
    }

    // Левый поворот
    Node<T>* rotateLeft(Node<T>* x) {
        // Меняем местами узлы
        Node<T>* y = x->right;
        Node<T>* T2 = y->left;
        y->left = x;
        x->right = T2;
        // Пересчитываем высоты
        x->height = max(height(x->left), height(x->right)) + 1;
        y->height = max(height(y->left), height(y->right)) + 1;
        return y;
    }

    // Рекурсивная вставка узла с балансировкой
    Node<T>* addAVL(Node<T>* node, T key) {
        // Стандартная вставка BST
        if (!node)
            return new Node<T>(key);
        if (key < node->key)
            node->left = addAVL(node->left, key);
        else
            node->right = addAVL(node->right, key);

        // Обновление высоты
        node->height = max(height(node->left), height(node->right)) + 1;

        // Проверка баланса
        int bf = balance(node);
```

*Рисунок 9 — дочерний класс **AVL** — часть 1*

```
397         // Проверка баланса
398         int bf = balance(node);
399
400         // Случаи вращения
401         // Проверка на границы уровня баланса и сравнение значений ключей узлов
402
403         // Лево
404         if (bf > 1 && key < node->left->key)
405             return rotateRight(node);
406         // Право
407         if (bf < -1 && key > node->right->key)
408             return rotateLeft(node);
409         // Лево/право
410         if (bf > 1 && key > node->left->key) {
411             node->left = rotateLeft(node->left);
412             return rotateRight(node);
413         }
414         // Право/лево
415         if (bf < -1 && key < node->right->key) {
416             node->right = rotateRight(node->right);
417             return rotateLeft(node);
418         }
419
420         return node;
421     }
422
423 protected:
424     // Переопределение виртуального метода добавления
425     Node<T>* add(Node<T>* node, T key) override {
426         return addAVL(node, key);
427     }
428 };
429
430 #endif
```

*Рисунок 10 — дочерний класс **AVL** — часть 2*

Было реализовано консольное меню, предоставляющее пользователю интерфейс для работы с программой (Рисунки 11-13).

```
7   int main() {
8       // Используем вещественные числа согласно варианту
9       AVL<double> tree;
10      int choice;
11      double val;
12      string filename;
13
14      // Для теста операторов
15      AVL<double> testTree;
16
17      while (true) {
18          cout << "\n============== MENU ==============\n";
19          cout << "1. Add node (keyboard input)\n";
20          cout << "2. Remove node\n";
21          cout << "3. Search node\n";
22          cout << "4. Display tree (visual)\n";
23          cout << "5. Tree traversals (PreOrder, InOrder, PostOrder)\n";
24          cout << "6. Execute Variant 5 Task\n";
25          cout << "7. Save to file\n";
26          cout << "8. Load from file\n";
27          cout << "9. Test operators (= and ==)\n";
28          cout << "0. Exit\n";
29          cout << "==================================\n";
30          cout << "Your choice: ";
31
32          cin >> choice;
33
34          if (cin.fail()) {
35              cin.clear();
36              cin.ignore(10000, '\n');
37              continue;
38          }
39
40          switch (choice) {
41          case 1: // Добавить узел
42              cout << "Enter value (double): ";
43              cin >> val;
44              tree.addValue(val);
45              cout << "Node added.\n";
46              break;
47          case 2: // Удалить узел
48              cout << "Enter value to remove: ";
49              cin >> val;
50              tree.removeValue(val);
51              cout << "Remove command executed.\n";
52              break;
53          case 3: // Найти узел
54              cout << "Enter value to search: ";
55              cin >> val;
56              if (tree.searchValue(val))
57                  cout << "Node FOUND.\n";
58              else
59                  cout << "Node NOT found.\n";
60              break;
61          case 4: // Отобразить дерево
62              cout << "\n--- Current Tree (operator<<) ---\n";
63              cout << tree; // Использование перегруженного оператора <<
64              break;
65          case 5: // Обходы дерева
66              cout << "PreOrder traversal: "; tree.preOrder();
67              cout << "InOrder traversal: "; tree.inOrder();
68              cout << "PostOrder traversal: "; tree.postOrder();
```

Рисунок 11 — Код меню – часть 1

```
67              cout << "InOrder traversal: "; tree.inOrder();        You, вчера • Прототип 4 лабы
68              cout << "PostOrder traversal: "; tree.postOrder();
69              break;
70          case 6:
71              tree.executeVariantTask(); // Выполняем задачу варианта 5
72              break;
73          case 7:
74              cout << "Enter filename: ";
75              cin >> filename;
76              tree.saveToFile(filename);
77              break;
78          case 8:
79              cout << "Enter filename: ";
80              cin >> filename;
81              tree.loadFromFile(filename);
82              break;
83          case 9:
84              cout << "\n--- Testing Operators ---\n";
85              testTree = tree; // Тест operator=
86              cout << "Tree copied to testTree via operator=.\n";
87              cout << "Comparison tree == testTree: " << (tree == testTree ? "EQUAL" : "NOT EQUAL") << endl;
88
89              cout << "Adding 999.9 to testTree...\n";
90              testTree.addValue(999.9);
91              cout << "Comparison tree == testTree: " << (tree == testTree ? "EQUAL" : "NOT EQUAL") << endl;
92              break;
93          case 0:
94              return 0;
95          default:
96              cout << "Invalid choice. Please try again.\n";
97          }
98      }
99
100     return 0;
101 }
```

Рисунок 12 — Код меню – часть 2

```
============== MENU ==============
1. Add node (keyboard input)
2. Remove node
3. Search node
4. Display tree (visual)
5. Tree traversals (PreOrder, InOrder, PostOrder)
6. Execute Variant 5 Task
7. Save to file
8. Load from file
9. Test operators (= and ==)
0. Exit
==================================
Your choice:
```

Рисунок 13 — Меню

Загрузим дерево из файла (Рисунки 14-16).

```
tree.txt U ×
C_codespace > 2 kurs > 3-rd senester > Algorithms and
1    2.3 2.2 2.1 2.2 5.2 2.4 10
```

Рисунок 14 — Загрузка из файла — часть 1

```
8. Load from file
9. Test operators (= and ==)
0. Exit
==================================
Your choice: 8
Enter filename: tree.txt
Tree loaded from file tree.txt
```

Рисунок 15 — Загрузка из файла — часть 2

```
4. Display tree (visual)
5. Tree traversals (PreOrder, InOrder, PostOrder)
6. Execute Variant 5 Task
7. Save to file
8. Load from file
9. Test operators (= and ==)
0. Exit
==================================
Your choice: 4

--- Current Tree (operator<<) ---
Root: 2.3
  Left node: 2.2 (Parent: 2.3)
    Left node: 2.1 (Parent: 2.2)
    Right node: 2.2 (Parent: 2.2)
  Right node: 5.2 (Parent: 2.3)
    Left node: 2.4 (Parent: 5.2)
    Right node: 10 (Parent: 5.2)
```

Рисунок 16 — Загрузка из файла — часть 3

Выполним обход дерева с выводом результатов (Рисунок 17).

```
5. Tree traversals (PreOrder, InOrder, PostOrder)
6. Execute Variant 5 Task
7. Save to file
8. Load from file
9. Test operators (= and ==)
0. Exit
==================================
Your choice: 5
PreOrder traversal: 2.3 2.2 2.1 2.2 5.2 2.4 10
InOrder traversal: 2.1 2.2 2.2 2.3 2.4 5.2 10
PostOrder traversal: 2.1 2.2 2.2 2.4 10 5.2 2.3
```

Рисунок 17 — Обход дерева

Выполним задачу **5** варианта, подразумевающую вывод значений узлов, соответствующих выбранному пользователем уровню (Рисунок 18).

```
6. Execute Variant 5 Task
7. Save to file
8. Load from file
9. Test operators (= and ==)
0. Exit
==================================
Your choice: 6
Enter the level to display nodes from: 2

--- Nodes at Level 2 ---
2.2 5.2
```

Рисунок 18 — Задание 5 варианта (вывод узлов по уровню)

Протестируем работу операторов копирования и сравнения (Рисунок 19).

```
9. Test operators (= and ==)
0. Exit
==================================
Your choice: 9

--- Testing Operators ---
Tree copied to testTree via operator=.
Comparison tree == testTree: EQUAL
Adding 999.9 to testTree...
Comparison tree == testTree: NOT EQUAL
```

Рисунок 19 — Вывод значений списка (Конструктор копирования)

Добавим новый узел в дерево (Рисунки 20-21).

```
1. Add node (keyboard input)
2. Remove node
3. Search node
4. Display tree (visual)
5. Tree traversals (PreOrder, InOrder, PostOrder)
6. Execute Variant 5 Task
7. Save to file
8. Load from file
9. Test operators (= and ==)
0. Exit
==================================
Your choice: 1
Enter value (double): 15.5
Node added.
```

Рисунок 20 — Добавление узла в дерево — часть 1

```
--- Current Tree (operator<<) ---
Root: 2.3
  Left node: 2.2 (Parent: 2.3)
    Left node: 2.1 (Parent: 2.2)
    Right node: 2.2 (Parent: 2.2)
  Right node: 5.2 (Parent: 2.3)
    Left node: 2.4 (Parent: 5.2)
    Right node: 10 (Parent: 5.2)
      Right node: 15.5 (Parent: 10)
```

Рисунок 21 — Добавление узла в дерево — часть 2

Удаление узла из дерева (Рисунки 22-23).

```
2. Remove node
3. Search node
4. Display tree (visual)
5. Tree traversals (PreOrder, InOrder, PostOrder)
6. Execute Variant 5 Task
7. Save to file
8. Load from file
9. Test operators (= and ==)
0. Exit
==================================
Your choice: 2
Enter value to remove: 10
Remove command executed.
```

Рисунок 22 — Удаление узла из дерева — часть 1

```
4. Display tree (visual)
5. Tree traversals (PreOrder, InOrder, PostOrder)
6. Execute Variant 5 Task
7. Save to file
8. Load from file
9. Test operators (= and ==)
0. Exit
==================================
Your choice: 4

--- Current Tree (operator<<) ---
Root: 2.3
  Left node: 2.2 (Parent: 2.3)
    Left node: 2.1 (Parent: 2.2)
    Right node: 2.2 (Parent: 2.2)
  Right node: 5.2 (Parent: 2.3)
    Left node: 2.4 (Parent: 5.2)
    Right node: 15.5 (Parent: 5.2)
```

Рисунок 23 — Удаление узла из дерева — часть 2

Найдём добавленный узел в дереве (Рисунок 24).

```
3. Search node
4. Display tree (visual)
5. Tree traversals (PreOrder, InOrder, PostOrder)
6. Execute Variant 5 Task
7. Save to file
8. Load from file
9. Test operators (= and ==)
0. Exit
==================================
Your choice: 3
Enter value to search: 15.5
Node FOUND.
```

Рисунок 24 — Поиск узла

Сохраним данные дерева в файл (Рисунки 25-26).

```
7. Save to file
8. Load from file
9. Test operators (= and ==)
0. Exit
==================================
Your choice: 7
Enter filename: tree2.txt
Tree saved to file tree2.txt
```

Рисунок 25 — Сохранение дерева в файл — часть 1

```
tree2.txt U ×
C_codespace > 2 kurs > 3-rd senester > Algorithms and
1   2.3 2.2 2.1 2.2 5.2 2.4 15.5
```

Рисунок 26 — Сохранение дерева в файл — часть 2

ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1. **Опишите общий принцип организации бинарного дерева поиска. В чем заключается преимущество подобной организации данных?**

Принцип: Бинарное дерево поиска — это иерархическая структура данных, состоящая из узлов. Каждый узел содержит:

1. **Ключ** (значение).
2. Указатель на **левое** поддерево.
3. Указатель на **правое** поддерево.

При этом все ключи левого поддерева меньше данного ключа, а у правого — больше.

Преимущество: поиск, вставка и удаление узлов выполняются относительно быстро, за время **O(h)**, где h — **высота дерева**.

2. **Опишите алгоритм построения бинарного дерева поиска?**
   1. Создать пустое дерево (корень = ***null***).
   2. Получить элемент входящей последовательности.
   3. Если дерево пустое, назначить данный элемент корнем дерева.
   4. Если дерево не пустое, рекурсивно спускаться по дереву от корня до конечного места для вставки (равному ***null***), выбирая между поддеревьями узлов по их значению относительно данного узла.
   5. Вставить входящий элемент на выбранную позицию (создать новый узел).
   6. Повторить шаги 2-5 для всех остальных элементов последовательности.
3. **Как выполняется поиск данных в бинарном дереве?**
   1. Начинаем цикл поиска с корневого узла.
   2. Сравниваем искомый ключ с ключом текущего узла.
   3. Если ключ совпал, элемент найден, поиск завершён.
   4. Если ключ не совпал, идём дальше по поддеревьям, в зависимости от разницы текущего ключа и искомого.
   5. Повторяем шаги 2-4, пока не уткнёмся в узел = ***null,*** т.е. данный ключ в дереве отсутствует.
4. **Опишите алгоритм удаления узла из бинарного дерева.**
   1. Найти узел нужный узел с помощью алгоритма поиска.
   2. Если узел не найден, завершить алгоритм удаления.
   3. Если узел найден, выбираем случай для удаления.
   4. Узел — это лист (без потомков): указатель родителя на ***nullptr***.
   5. Узел имеет одного потомка: указатель родителя на потомка заменяется теперь указывает на потомка текущего узла.
   6. Узел имеет двух потомков: находим минимальный элемент в правом поддереве или максимальный в левом для данного узла; скопировать значение выбранного элемента в удаляемый узел; удалить сам узел с минимальным значением (он будет иметь не более 1 потомка, поэтому применяем шаги 4 или 5).

5. **Перечислите и охарактеризуйте различные способы обхода бинарного дерева.**
    - **Прямой (префиксный):** Корень -> Левое поддерево -> Правое поддерево. Полезно для создания копии дерева.
    - **Обратный (инфиксный):** Левое поддерево -> Корень -> Правое поддерево. Для BST всегда выдаёт элементы в отсортированном порядке.
    - **Концевой (постфиксный):** Левое поддерево -> Правое поддерево -> Корень. Полезен для удаления дерева.

ВЫВОДЫ

В ходе выполнения лабораторной работы были изучены принципы организации и обработки бинарных деревьев, а также его модификации — AVL-дерева. Был изучен метод рекурсивной обработки данных.